



NORTHERN UNIVERSITY OF BUSINESS & TECHNOLOGY KHULNA

Department of Computer Science and Engineering



SMART FARMER ASSISTANT

AI-Based Agriculture Support System



Submitted To:

Md. Riaz Mahmud

Assistant Professor, Department of Computer Science and Engineering

Northern University of Business and Technology Khulna

Submitted By:

Team Name: CSE4204-8D-T03

Team Leader: Sakib Foysal Ejarder

ID: 11220320948

Section: 8D

Submission date: 13th August 2026

CSE4204 — Mobile Computing Lab

Week 08 Submission

AI Workflow Report

Smart Farmer Assistant

AI-Based Agriculture Support System

Table of Contents

Content	Page
1. Team Information	2
2. Purpose of the AI Workflow	2
3. Overall System Workflow	3
4. Shared Security and Reliability Controls	4
5. Workflow A — Smart Farmer AI Chatbot	5
6. Workflow B — Crop-Leaf Visual Assessment	7
7. Workflow C — Fertilizer Planning Guide	10
8. Error-Handling Workflow	13
9. Prompt-to-Output Validation Map	14
10. Files That Implement the Workflow	14

1. Team Information

Field	Details
Team Name	CSE4204-8D-T03
Section	8D
Course	CSE4204 — Software Engineering Lab
Semester	Spring 2026
Institution	Leading University / University (Bangladesh)
GitHub Repo	github.com/sakib-foysal/CSE4204-8D-T03-SmartFarmer-Assistant

1.1 Team Members

#	Name	Student ID	Role
1	Sakib Foysal Ejarder	11220320948	Team Leader / Backend
2	Md. Noyon Sheikh	11220320946	Frontend Developer
3	Sayed Tauhidul Islam	11220320950	AI / ML Engineer
4	Sayed Akib Osman	11220320974	Database / QA

2. Purpose of the AI Workflow

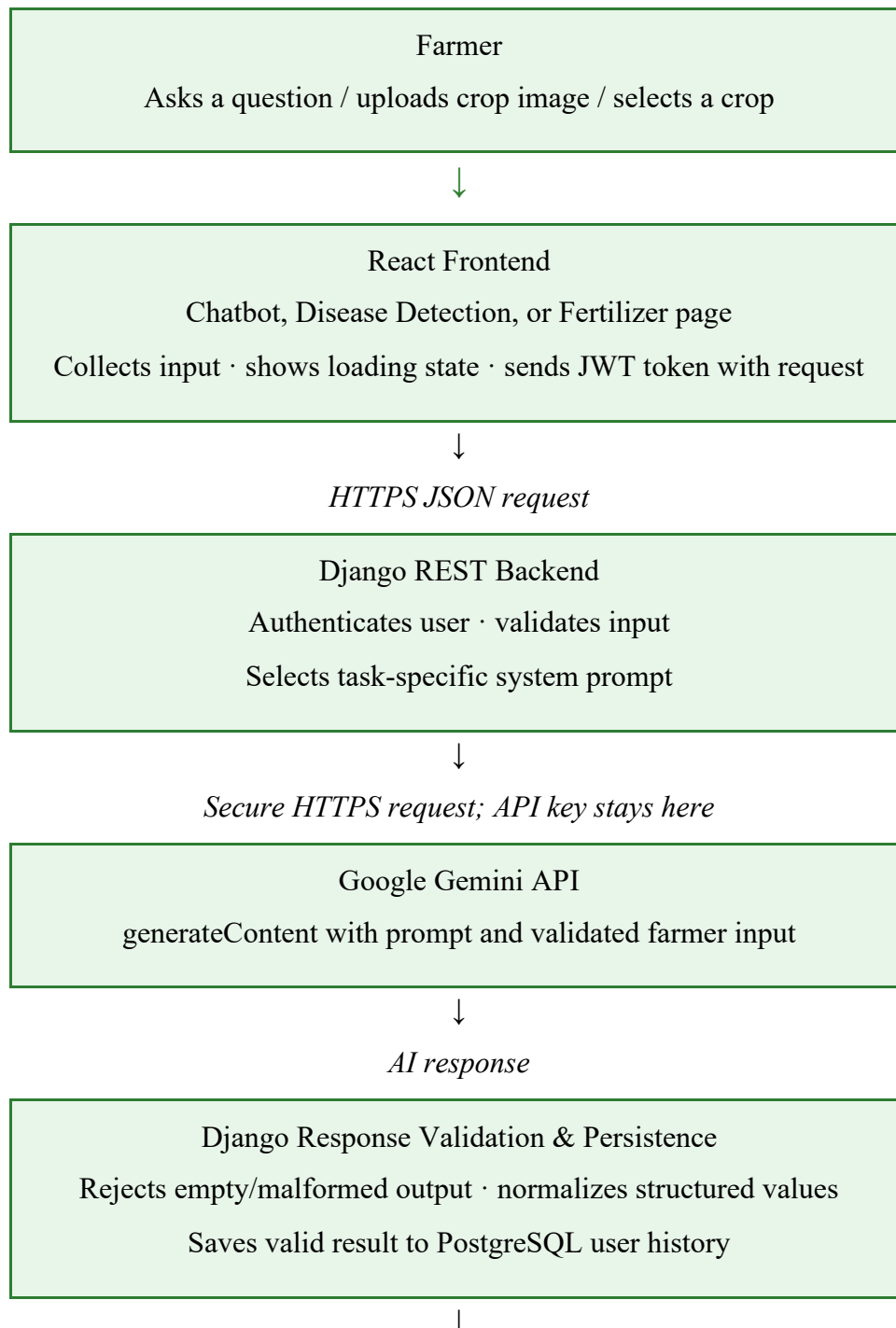
This workflow explains how Smart Farmer Assistant safely turns a farmer's input into a useful AI-generated result. The system supports three connected AI features:

1. Smart Farmer AI Chatbot — answers farming questions in Bangla, English, or mixed language.
2. Crop-Leaf Visual Assessment — examines an uploaded crop/leaf image and provides a cautious visual assessment.
3. Fertilizer Planning Guide — generates a crop-specific planning guide in structured JSON.

The design keeps the Gemini API key on the Django backend, validates input before an AI call, validates output before it reaches the user, saves only successful results, and keeps the application usable when the AI service is unavailable.

3. Overall System Workflow

The diagram below traces a single request from the farmer through the frontend, backend, Gemini API, and back to the screen.



JSON success or safe error

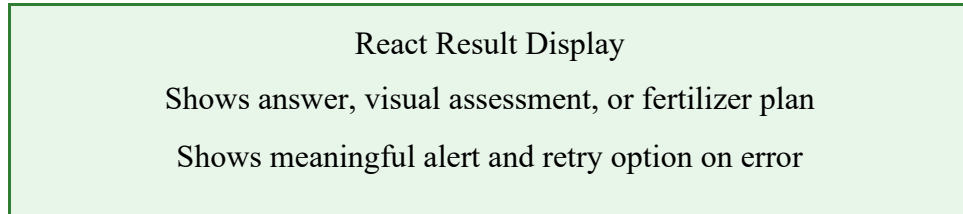


Figure 1. Overall system workflow — from farmer input to displayed AI result.

4. Shared Security and Reliability Controls

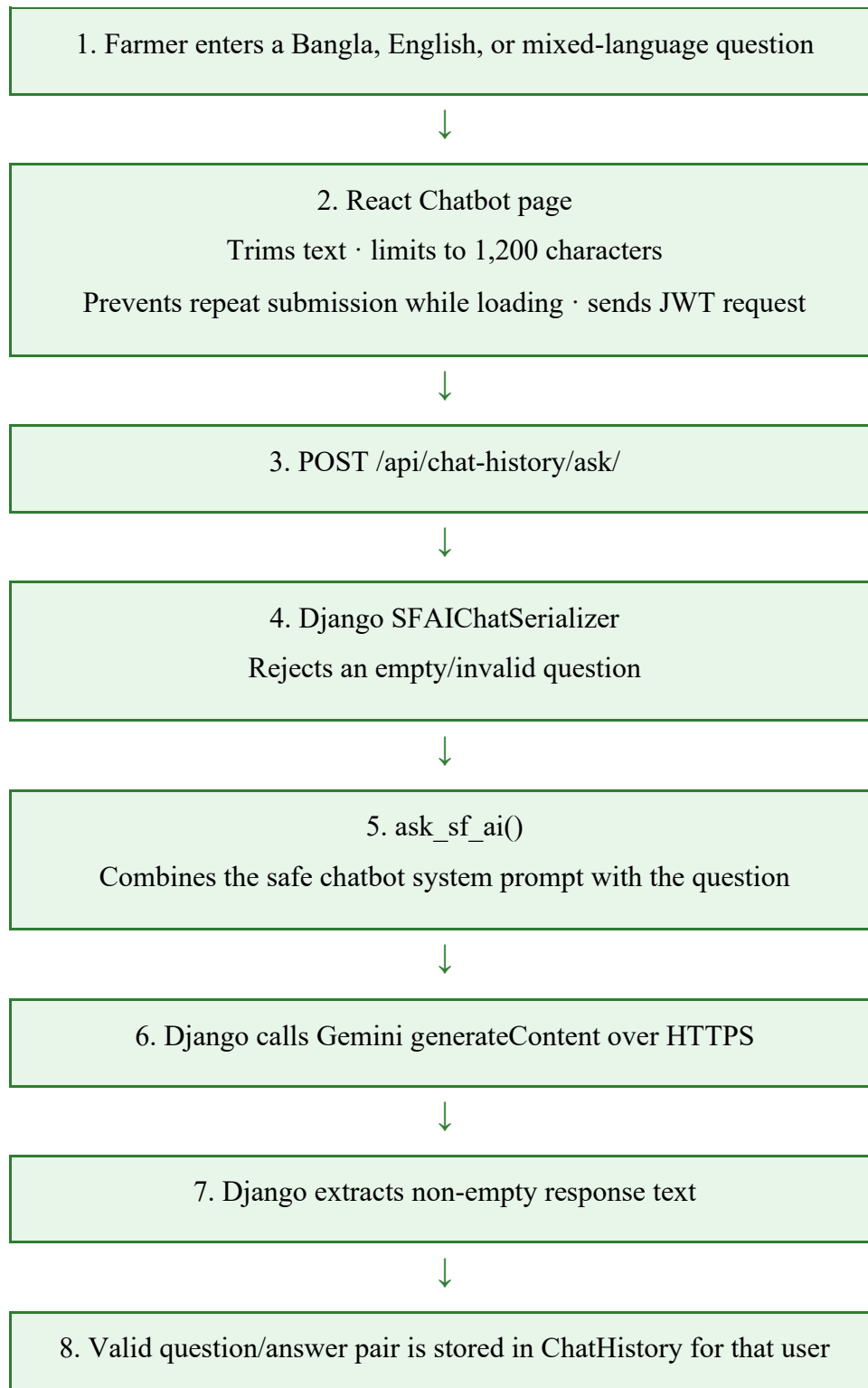
Control	How It Works
Authentication	All AI endpoints require the application's JWT authentication permission.
API-key protection	GEMINI_API_KEY is read on the Django server from backend/.env; it never goes to the browser.
Transport security	Django calls Gemini over HTTPS.
Input validation	Serializers validate required fields, length, image type, base64 data, and image size before the AI request.
Timeout	Gemini requests time out after 25 seconds by default.
Retries	Only transient connection/server failures are retried; TLS verification remains enabled.
Output validation	Empty, invalid JSON, malformed, or wrong-shape results are rejected before display.
Data isolation	Histories are filtered by the authenticated user, so a farmer receives only their own saved results.
Safe language	Prompts avoid certain diagnoses, fabricated live information, and universal chemical-dose claims.

5. Workflow A — Smart Farmer AI Chatbot

User goal

A farmer needs immediate first-response guidance about a crop, pest, disease, irrigation, fertilizer, weather preparedness, or market planning.

Technical flow





9. JSON response returns to React and appears in the conversation

Figure 2. Technical flow of the Smart Farmer AI Chatbot feature.

API contract — Request

POST /api/chat-history/ask/

Authorization: Bearer <JWT_TOKEN>

Content-Type: application/json

```
{  
  "question": "আমার ধানের পাতায় বাদামি দাগ হচ্ছে, কী করব?"  
}
```

API contract — Successful response

```
{  
  "id": "saved-history-id",  
  "question": "আমার ধানের পাতায় বাদামি দাগ হচ্ছে, কী করব?",  
  "response": "AI-generated, safety-aware farming guidance",  
  "date": "saved timestamp",  
  "model": "SF AI"  
}
```

The user-facing response label in the current code is “SF AI”; the actual configured provider is Google Gemini. The API key and model configuration use GEMINI_API_KEY and GEMINI_MODEL.

AI Chatbot

An AI farming assistant for your questions in Bangla or English

Smart Chatbot

Live SF AI farming guidance in Bangla or English

আমার ধানের পাতায় বাদামি দাগ হচ্ছে, কী করব?



1. সম্ভাব্য কারণ: ধানের পাতার বাদামি দাগ Brown Spot রোগ হতে পারে, তবে ব্লাস্ট, ব্যাকটেরিয়াজনিত রোগ বা পুষ্টির ঘাটতিও একই রকম দেখাতে পারে। শুধু লেখা দেখে নিশ্চিত রোগ বলা নিরাপদ নয়।
2. এখনই করণীয়: বেশি আক্রান্ত পাতা/গাছের অংশ আলাদা করে জমির বাইরে নষ্ট করুন। জমিতে পানি জমে থাকলে নালা দিয়ে বের করুন এবং কয়েক দিন অতিরিক্ত ইউরিয়া দেবেন না।
3. ভালোভাবে দেখুন: দাগগুলো গোল/ডিম্বাকার ও মাঝখান হালকা হলে Brown Spot-এর সম্ভাবনা বাড়ে। দাগ দ্রুত ছড়ালে বা চারার বড় অংশ আক্রান্ত হলে আজই ছবি তুলে রাখুন।
4. প্রতিরোধ: পরিষ্কার বীজ ব্যবহার, সুস্থ সার, জমির আগাছা ও আক্রান্ত অবশিষ্টাংশ পরিষ্কার

Ask a farming question...

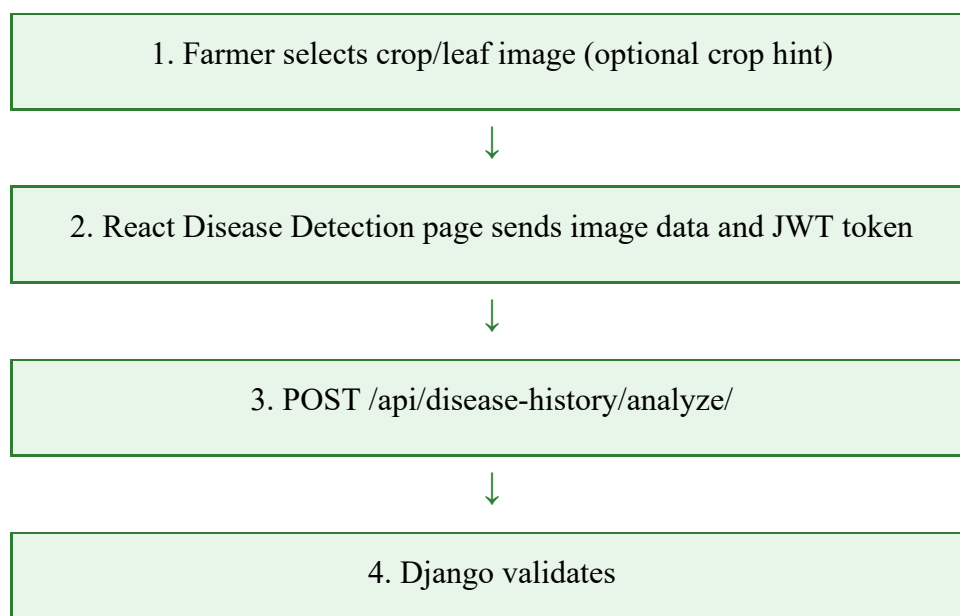
Figure 3: API Contract

6. Workflow B — Crop-Leaf Visual Assessment

User goal

A farmer uploads a leaf/crop image to receive a cautious first visual assessment and recommended next checks.

Technical flow



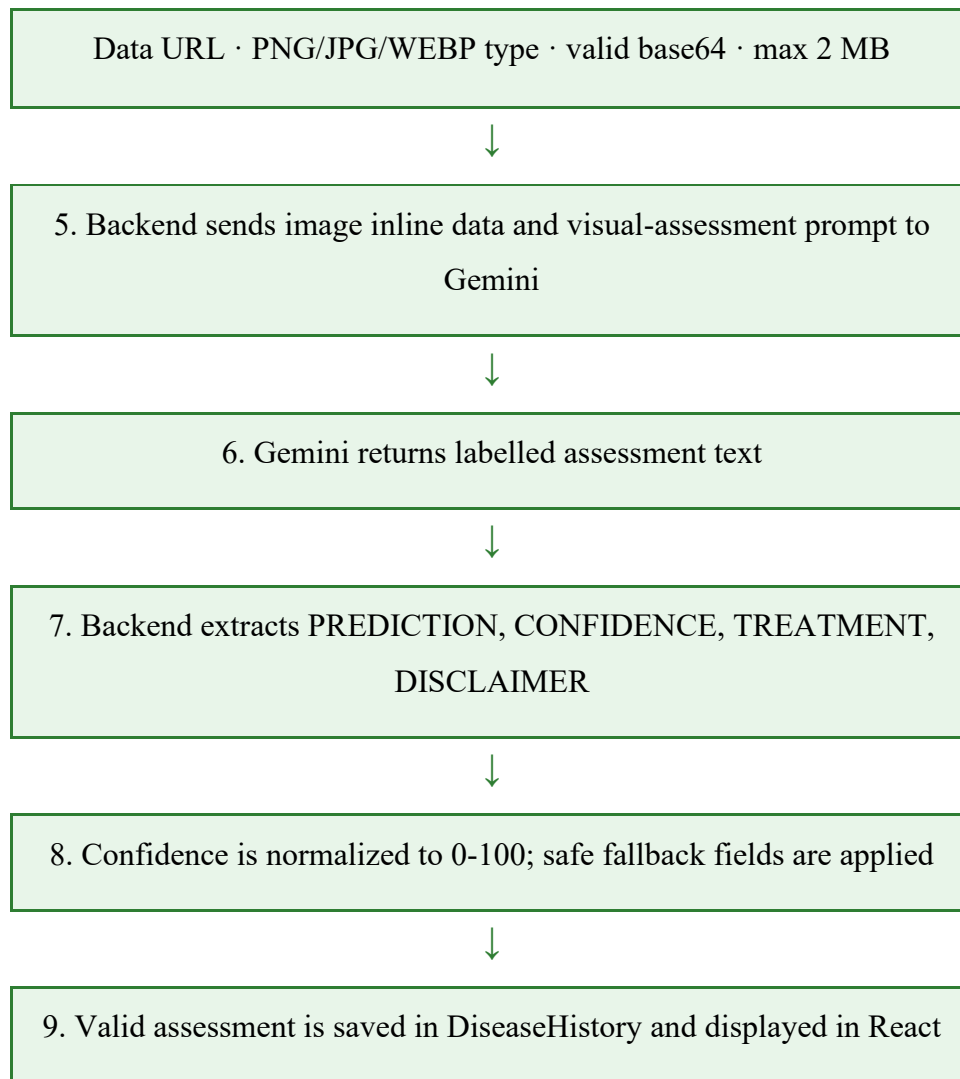


Figure 4. Technical flow of the Crop-Leaf Visual Assessment feature.

API contract — Request body

```
{
  "image_data": "data:image/jpeg;base64,<encoded-image-data>",
  "crop_hint": "Tomato"
}
```

API contract — Successful response shape

```
{
  "id": "saved-history-id",
  "prediction": "Likely visible issue",
  "confidence": 0,
```

```
"treatment": "Safe next steps",  
"disclaimer": "This is an AI visual assessment, not a laboratory-confirmed diagnosis.",  
"date": "saved timestamp"  
}
```

Safety rule

The prompt requires the model to assess only what is visible. The interface labels confidence as visual confidence and always shows a disclaimer. The system does not claim to provide laboratory confirmation.

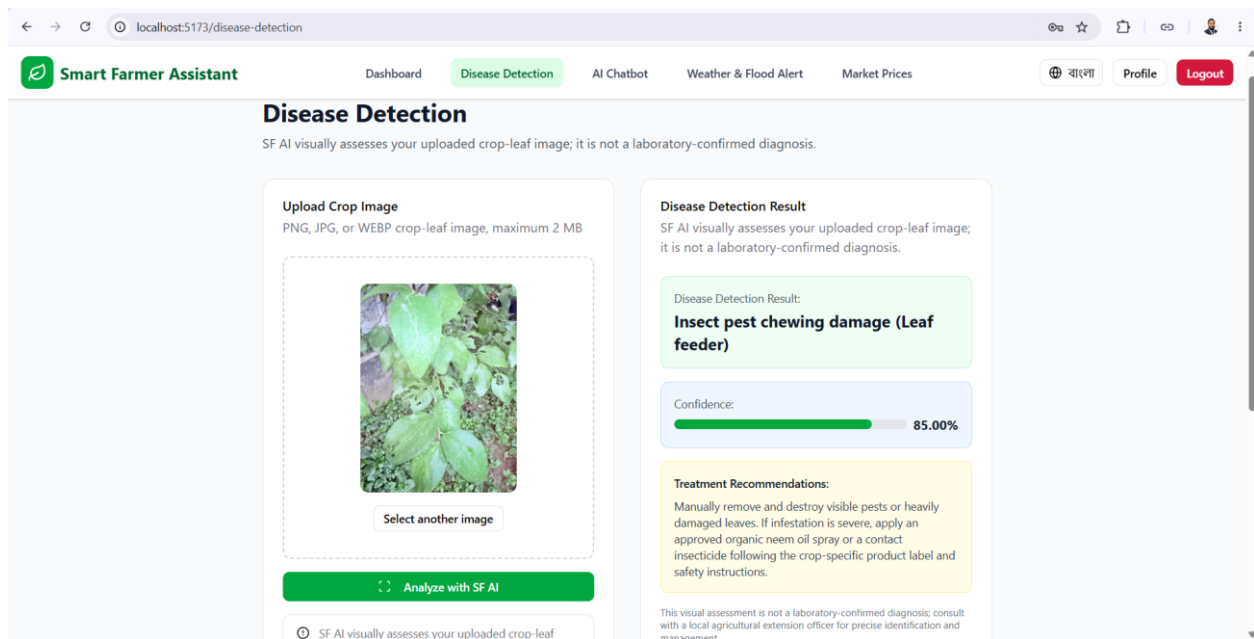


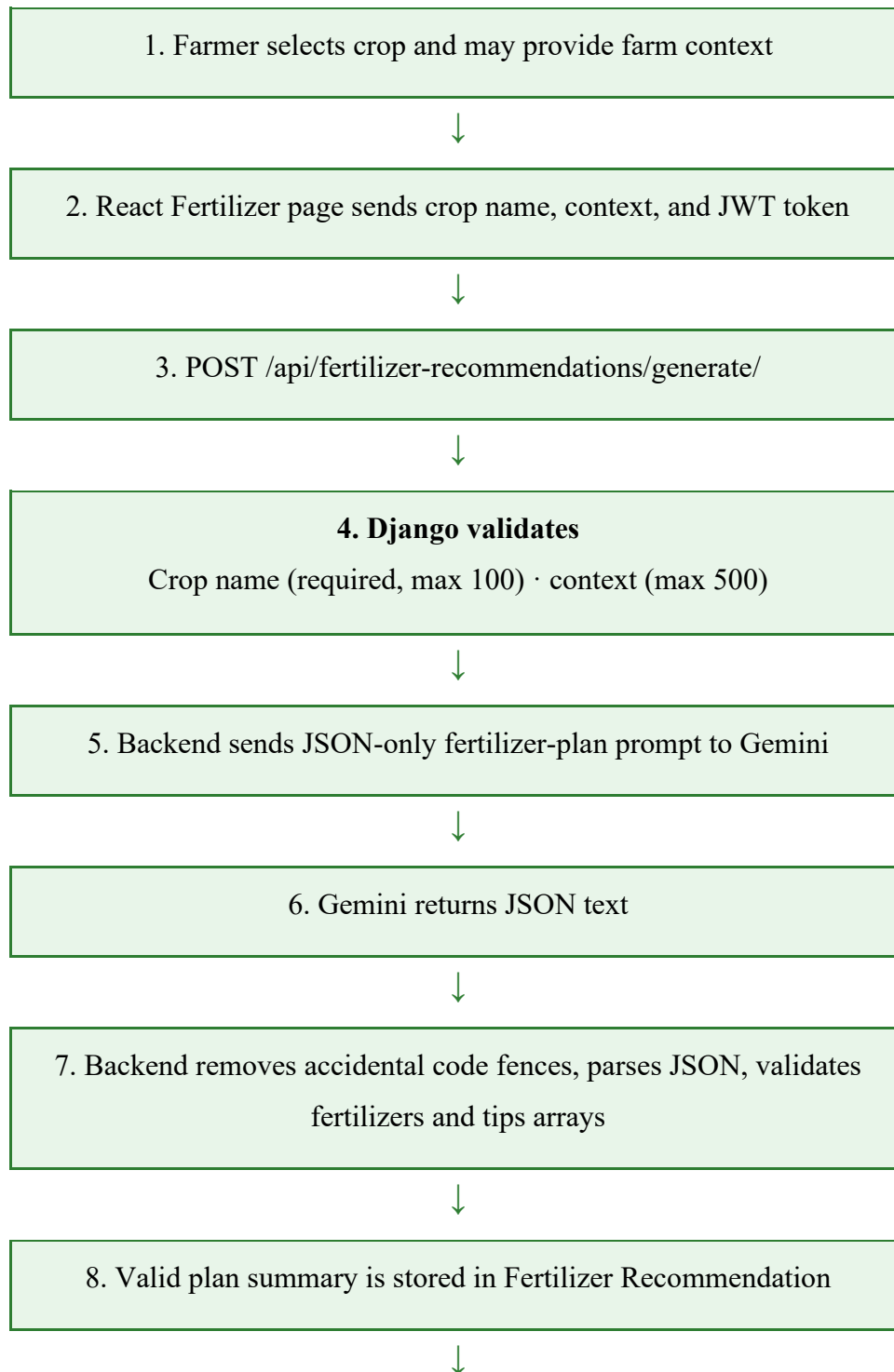
Fig 5: Disease Detection

7. Workflow C — Fertilizer Planning Guide

User goal

A farmer selects a crop and receives a dynamic planning guide rather than a fixed recommendation card.

Technical flow



9. React renders planning items, tips, and mandatory disclaimer

Figure 6: Technical flow of the Fertilizer Planning Guide feature.

API contract — Request body

```
{
  "crop_name": "Rice",
  "farm_context": "Optional farmer-provided context"
}
```

API contract — Expected successful response shape

```
{
  "id": "saved-plan-id",
  "crop": "Rice",
  "fertilizers": [
    { "name": "Fertilizer name", "amount": "General planning guidance", "timing": "Suggested stage" }
  ],
  "tips": ["Simple application tip"],
  "disclaimer": "Confirm dosage with a soil test, local agriculture officer, and product label.",
  "date": "saved timestamp"
}
```

Safety rule

The system prompt requires general planning guidance, not a universal dose prescription. It explicitly tells the farmer to confirm any dosage with a soil test, local agricultural-extension officer, and product label.

Fertilizer Recommendation

AI-generated guide; not a universal fertilizer prescription

Selected crop

Crop shown in the uploaded image

Balanced fertilizer plan

Use a balanced NPK fertilizer only according to a soil test and the product label.

Apply in split doses during land preparation and the crop's growth stages.

Organic matter

Use well-decomposed compost or manure according to local guidance.

Incorporate it well into the soil during land preparation.

- ✓ Do not apply too much fertilizer at once; split applications according to crop growth.
- ✓ Irrigate as needed after fertilizer application, without allowing standing water.
- ✓ Avoid excessive nitrogen while the crop has active pest or disease damage.

 This is a safe general guide. Confirm the exact dose with a soil test, crop stage, and product label.

Figure 7: Fertilizer Recommendation

8. Error-Handling Workflow

The decision flow below shows how the backend responds to invalid input or AI-service problems, so the application keeps working even when Gemini is unavailable.

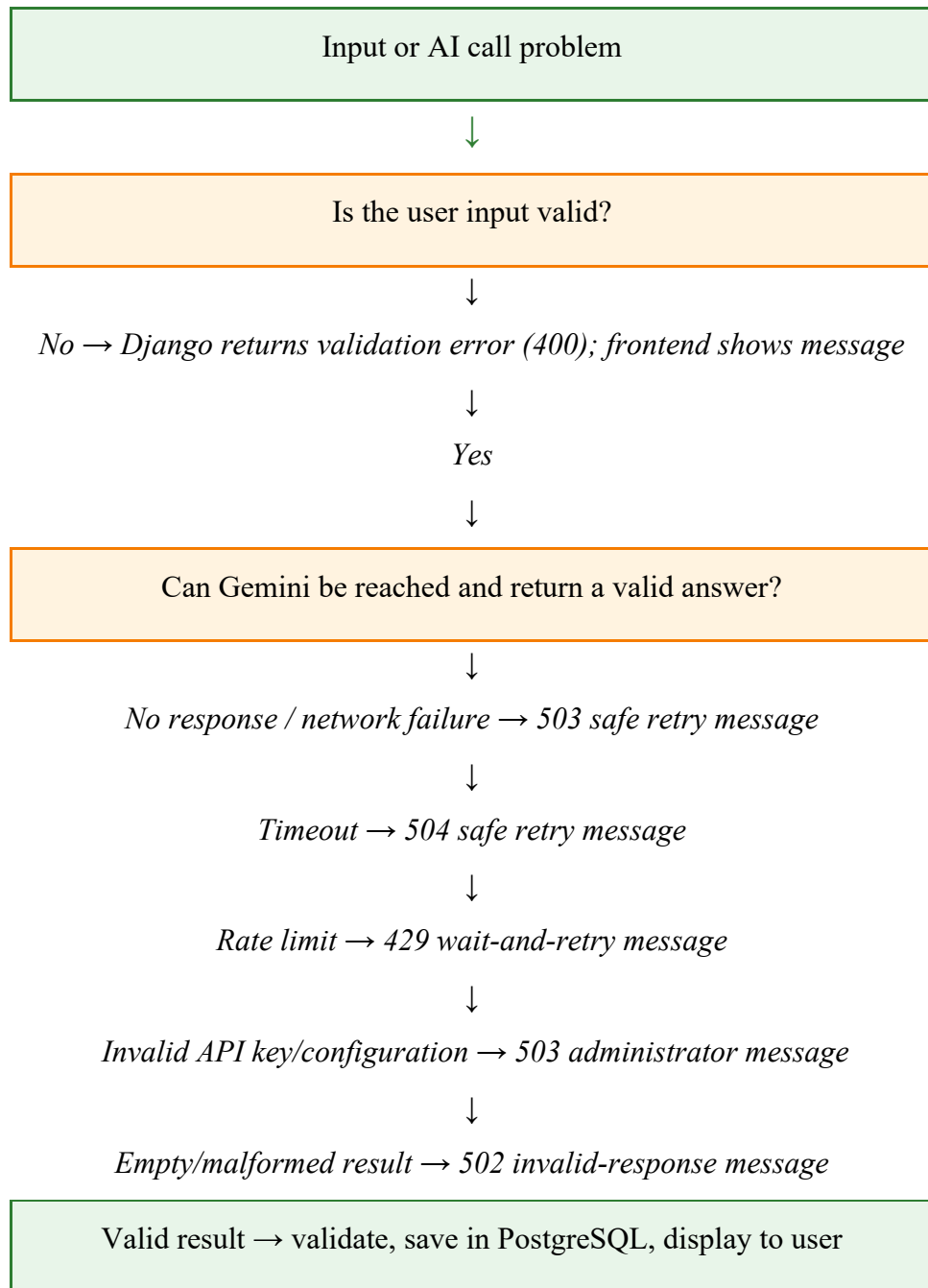


Figure 8: Error-handling decision flow for invalid input and AI-service failures.

The frontend catches failed requests and renders an alert. It does not crash, fabricate an answer, or store an unsuccessful response. The farmer can continue to use the non-AI parts of the application and retry later.

9. Prompt-to-Output Validation Map

Feature	Required Prompt Behavior	Backend Validation Before Display
Chatbot	Same language; short practical answer; no certain diagnosis; local guidance reminder	Non-empty extracted text required
Visual assessment	Exact labelled fields; visual-only claim; no exact chemical dose	Extracts labels; confidence normalized to 0-100; fallback disclaimer applied
Fertilizer plan	JSON only with crop, fertilizers, tips, disclaimer; no universal dose	Strips code fence if needed; parses JSON; requires fertilizers and tips arrays

10. Files That Implement the Workflow

Layer	Key Implementation File	Responsibility
AI client and prompts	backend/apps/chatbot/sf_ai_service.py	Gemini call, prompts, retries, timeout, response parsing
Chat endpoint	backend/apps/chatbot/views.py	Validates question, calls AI, saves ChatHistory
Image endpoint	backend/apps/disease_detection/views.py	Validates image assessment and saves DiseaseHistory
Fertilizer endpoint	backend/apps/fertilizer/views.py	Validates JSON plan and saves recommendation history

Layer	Key Implementation File	Responsibility
Input serializers	backend/apps/{chatbot,disease_detection,fertilizer}/serializers.py	Enforces input rules
API settings	backend/config/settings.py	Reads GEMINI_MODEL, timeout, CORS configuration
Chat interface	frontend/src/app/pages/user/ChatbotPage.tsx	Sends question, displays answer/loading/error/history
Image interface	frontend/src/app/pages/user/DiseaseDetectionPage.tsx	Uploads image and displays assessment/disclaimer
Fertilizer interface	frontend/src/app/pages/user/FertilizerPage.tsx	Requests and renders dynamic fertilizer plan
Frontend API client	frontend/src/app/lib/api.ts	Sends authenticated JSON requests and converts API errors to messages